



ROBOTRONICS

Indian Institute of Technology Mandi

Smart Key Storage System

Web Development Architecture & Implementation Report

Project: Smart Key Storage (SKSS)	Module: Web Application (FE + BE)
Site: SAC, Room A-19	Club: Robotronics, IIT Mandi

1 Introduction

1.1 Problem Statement

Retrieving a clubroom key at the IIT Mandi Student Activity Center currently requires a club coordinator to be physically present with a master keycard. When the coordinator is unavailable, members are locked out of otherwise permitted, routine access. This creates scheduling friction for every club sharing the space and places an unnecessary availability burden on coordinators.

1.2 Proposed Solution

The Smart Key Storage System (SKSS) replaces coordinator-mediated handoff with authenticated, self-service key retrieval through a purpose-built web application, backed by an IoT-enabled key enclosure. This document focuses specifically on the **software layer** of the system — the web application, backend services, data architecture, and their integration with the physical enclosure.

1.3 Objectives of the Web Application

- Allow any permitted club member to retrieve or return a room key in under 30 seconds.
- Enforce two-factor authentication and on-site proximity verification before any physical action is triggered.
- Maintain a complete, tamper-evident audit trail of every digital and physical access event.
- Provide a unified admin/coordinator dashboard for access control, live monitoring, and device health.

2 System Architecture Overview

2.1 Component Stack

2.2 Design Principle: Why Not Host Everything on the ESP32

The ESP32-S3 comfortably serves the captive-portal redirect page, but it cannot host password/TOTP-grade authentication, a relational audit database, SMTP delivery, or a real-time dashboard usable from anywhere on campus. Keeping these on a proper backend (FastAPI + PostgreSQL + Redis) satisfies the actual requirement — *physical proximity for key actions* — without sacrificing remote monitoring, which coordinators and admins need daily.

3 Frontend Architecture (Next.js)

Layer	Component	Technology
Backend	API server	FastAPI (Python 3.12)
Backend	Primary datastore	PostgreSQL (Supabase)
Backend	Cache / pub-sub / rate limiting	Redis (Upstash)
Backend	Device message broker	MQTT (HiveMQ Cloud / Mosquitto)
Frontend	User web app & Admin panel	Next.js 14 (role-gated routes in same app)
Notification	Email service	SMTP, provider-pattern module

Table 1: Web/software technology stack

3.1 Application Structure

The frontend is a single Next.js 14 application serving two role-gated surfaces:

- **User Web App** — login, live key-status grid, retrieve/return actions, possession timer, extension requests.
- **Admin Panel** — user/permission management, live monitoring, audit logs, device health, notification configuration, reports.

3.2 Key UX States

State	Behaviour
Not logged in	Login + key-status grid browsing allowed from any network
Logged in, no proximity	Retrieve / Return / Start-session buttons disabled; “Connect to enclosure WiFi” prompt shown
Logged in, proximity-verified	Full retrieve/return actions enabled for 5 minutes
Key retrieved	Live countdown to <code>due_at</code> , extend option surfaced before expiry

Table 2: Frontend state matrix

3.3 Admin Panel Views

1. **Dashboard** — keys out, overdue count, device health, unresolved tamper flags.
2. **User management** — create/edit/deactivate accounts (admin-only).
3. **Permission management** — grant/revoke access (coordinator: own room; admin: all).
4. **Room & key management** — map slot to room, per-slot maintenance mode.
5. **Live monitoring** — real-time “who holds what” board.
6. **Audit logs** — access, retrieval, and tamper logs with CSV export.
7. **Device health** — battery status, RSSI, last heartbeat, offline thresholds.

4 Backend Architecture (FastAPI)

4.1 Service Breakdown

Service	Responsibility
Auth service	Credential login, TOTP enroll/verify, session issuance
Proximity service	Validates device-issued proximity codes, sets/checks verification flag
Key service	Retrieve/return/extend, permission checks, MQTT command dispatch
Admin service	User/room/permission CRUD, logs, device health
MQTT listener	Background worker; subscribes to device topics, writes DB, pushes Web-Socket updates
Notification service	Retrieval confirmation, reminders, warnings, and escalation emails

Table 3: Backend service responsibilities

4.2 Data Stores

- **PostgreSQL (Supabase)** — source of truth: users, permissions, key/slot state, all audit logs.
- **Redis (Upstash)** — session cache, login rate-limiting, proximity codes (120s TTL), live status board without polling the DB.

5 Database Schema

5.1 Core Entities & Sample DDL

The database tracks keys, users, and logs. Below is the fundamental SQL setup:

```
CREATE TYPE user_role AS ENUM ('member', 'coordinator', 'admin');
CREATE TYPE key_status AS ENUM ('available', 'retrieved', 'maintenance');

CREATE TABLE users (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  email text UNIQUE NOT NULL,
  role user_role NOT NULL DEFAULT 'member',
  password_hash text NOT NULL,
  totp_secret text,
  is_active boolean NOT NULL DEFAULT true
);

CREATE TABLE key_slots (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  device_id uuid NOT NULL REFERENCES devices(id),
  slot_number smallint NOT NULL CHECK (slot_number BETWEEN 1 AND 8),
  room_id uuid REFERENCES rooms(id),
  status key_status NOT NULL DEFAULT 'available'
);
```

5.2 Concurrency Safety

Retrieval uses an atomic conditional update rather than check-then-set, preventing race conditions:

```
UPDATE key_slots SET status = 'retrieved'
WHERE id = $1 AND status = 'available';
-- If affected row count is 0, the backend returns "409 slot_unavailable"
```

6 API Design

6.1 Representative Endpoints

Method	Path	Auth	Description
POST	/proximity/verify	device code	Validate enclosure code
POST	/auth/login	none	Credentials → TOTP challenge
POST	/auth/totp/verify	partial	Verify TOTP → session token
POST	/sessions/start	user + proximity	Begin session, publish unlock
GET	/keys	user	List slots + status
POST	/keys/{slot_id}/retrieve	user + proximity	Permission check → dispense
POST	/keys/{slot_id}/return	user + proximity	Unlock for reinsertion
POST	/keys/{slot_id}/extend	user	Extend <code>due_at</code>
GET	/admin/logs/override	admin/coordinator	Tamper/override log query

Table 4: Core API surface

6.2 Sample Exchange — Key Retrieval

```

POST /keys/{slot_id}/retrieve
{"session_id": "b1e7..." }

200OK
{
  "slot_id": "3f2a...",
  "status": "retrieved",
  "due_at": "2026-08-07T18:30:00Z",
  "retrieval_log_id": "9c11..."
}

```

7 Authentication, Proximity & Security

7.1 Two-Factor Authentication

- Password + TOTP (RFC 6238, 30s step, ± 1 step tolerance).
- Session tokens are server-side and revocable via Redis. If a phone is reported lost, an admin can instantly terminate the active sessions.
- Rate limiting blocks users after 5 failed login or TOTP attempts.

7.2 Proximity Verification Protocol

To execute physical key actions, a user's browser session must prove on-site proximity:

1. User connects to the Access Controller's physical WiFi AP.
2. Controller generates an 8-character random code and publishes it over MQTT.
3. Backend caches this code in Redis mapped to the device with a 120s TTL.
4. User is redirected to the web app, which calls `POST /proximity/verify`.
5. If verified, the session receives a 5-minute proximity flag allowing retrieve or return commands.

7.3 Security Controls Summary

- TLS encryption on all API and MQTT traffic.
- Per-controller unique MQTT credentials — no shared passwords.
- Single-use nonces on commands protect against physical or replay attacks.
- TOTP secrets are stored encrypted at rest.

8 Real-Time & Notification Systems

8.1 Live Status Updates

The background MQTT worker captures physical key actions, updates PostgreSQL, and simultaneously triggers Redis pub/sub. Connected browsers receive live updates over WebSockets, ensuring status views stay perfectly updated.

8.2 Notification Schedule

Event	Timing	Recipient
Retrieval confirmation	Immediate	Member
Return reminder	T-30min before due	Member
Overdue warning	At due time	Member
Coordinator escalation	T+2h past due	Coordinator

Table 5: Default notification timing

The notification engine utilizes the existing internal SMTP library module, enabling clean template dispatch and modular extension if providers change.

9 Deployment & Scalability

9.1 Deployment Targets

Component	Host	Rationale
FastAPI backend	Railway / Fly.io	Simple deploy, cheap at pilot scale
PostgreSQL	Supabase	Managed database, generous free tier
Redis	Upstash	Serverless-friendly, instant scaling
Next.js apps	Vercel	Edge routing, zero-config, free tier
MQTT broker	HiveMQ Cloud	Reliable cloud broker, free tier

Table 6: Deployment stack summary

9.2 Scalability Notes

The schema natively supports multiple enclosures via `devices` and `key_slots.device_id` associations. FastAPI being completely stateless allows adding edge instances behind a simple load balancer if usage scales.

10 Conclusion

The web development layer of the Smart Key Storage System delivers authenticated, proximity-gated, self-service key access while preserving a complete audit trail. Built entirely on free-tier, well-supported, and scalable infrastructure, the architecture is designed to scale from a single pilot enclosure at SAC A-19 to a multi-enclosure deployment without structural rework.